

Duplicate checker

(ver 2)

	0	1	2	3	4	5
A	1	2	3	4	4	5

$3(3)$
2

B	1	2	3	4	5	6
	0	1	2	3	4	5

For A

1) $i = 0$ - $0 < 6$ (length)

$j = 0 + 1$ - $1 < 6$

$a[0] == a[1]$

$1 == 2$ false $\times$

2) $i = 1$ $1 < 6$

$j = 2$ $2 < 6$

$a[1] == a[2]$

$2 == 3$ - false

3) $i = 2$ $2 < 6$

$j = 3$ $3 < 6$

$a[2] == a[3]$

$3 == 4$ - false

4) $i = 3$ $3 < 6$

$j = 4$ $4 < 6$

$a[3] == a[4]$

$4 == 4$ - true

~~$\times$ end p~~

Medicare Pvt. Ltd.

for B

1	2	3	4	5	6
---	---	---	---	---	---

0 1 2 3 4 5

1) $i=0$ $0 < 6$
 $j=1$ $1 < 6$
 $a[0] == a[1]$
 $1 == 2$ — false

2) $i=1$ $1 < 6$
 $j=2$ $2 < 6$
 $a[1] == a[2]$
 $2 == 3$ — false

3) $i=2$ $2 < 6$
 $j=3$ $3 < 6$
 $a[2] == a[3]$
 $3 == 4$ — false

4) $i=3$ $3 < 6$
 $j=4$ $4 < 6$
 $a[3] == a[4]$
 $4 == 5$ — false

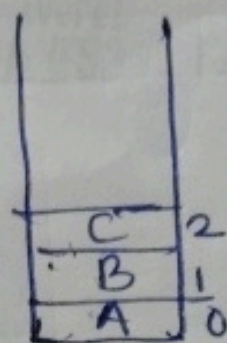
5) $i=4$ $4 < 6$
 $j=5$ $5 < 6$
 $a[4] == a[5]$
 $5 == 6$ — false

6) $i=5$ $5 < 6$
 $j=6$ $6 < 6$ — false
 — end —

→ no duplicate found

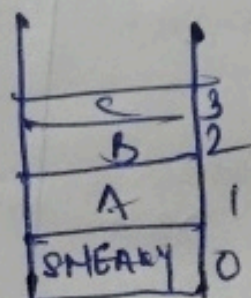
Stack Flow (part A)

s.push('A')
s.push('B')
s.push('C')
s.add(0, "SNEAKY")

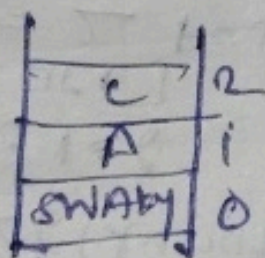


Stack S.

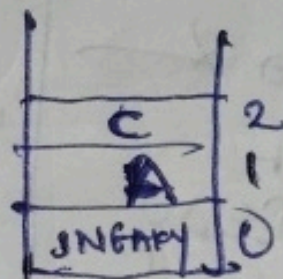
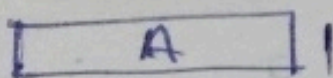
s.remove(2)



sop(s)



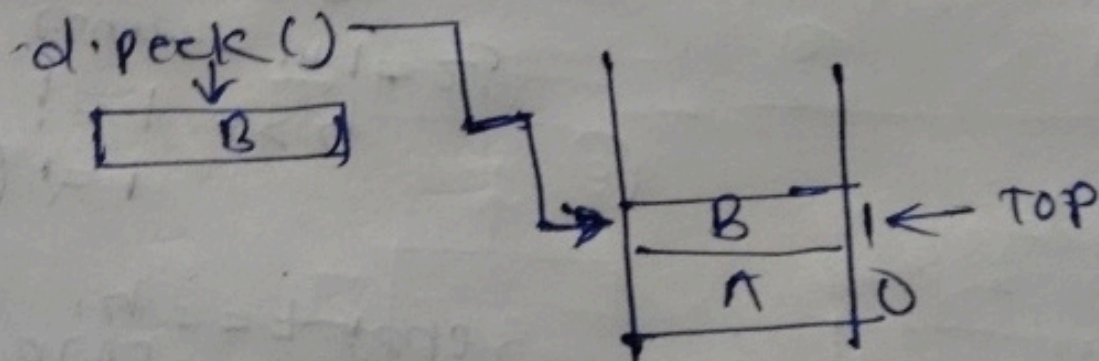
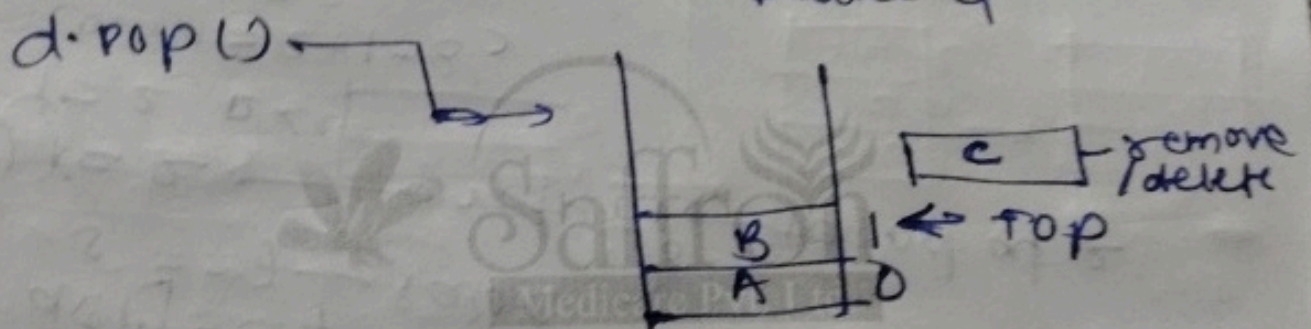
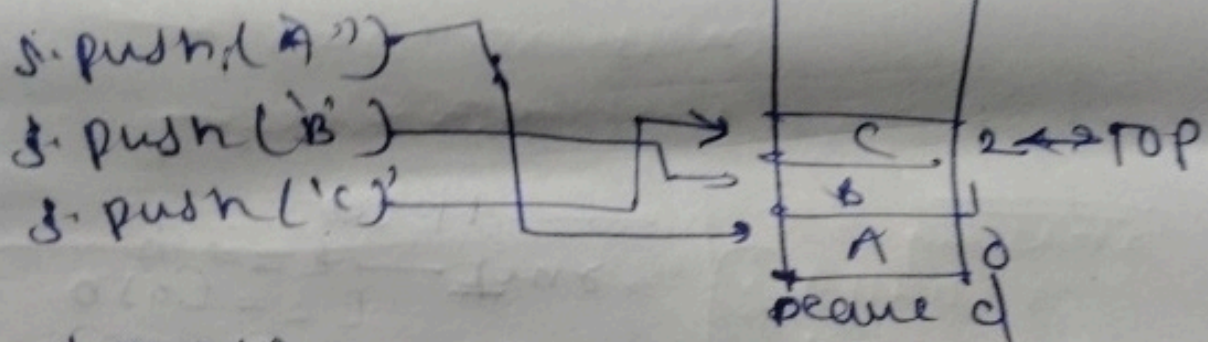
getu



Stack Flow part B

s.push('A')

Stack Flow part B



Linear Search

1) count = 0 ~~1~~ ~~2~~ ~~3~~ key = 7

0	1	2	3
4	5	7	10

2) for i

i = 0 $0 < 4$

count = 0 + 1 = 1

a[0] == 7

4 == 7 false

2) i = 1

i = 1 $1 < 4$

count = 1 + 1 = 2

a[1] == 7

5 == 7 false

3) i = 2

i = 2 $2 < 4$

count = 2 + 1 = 3

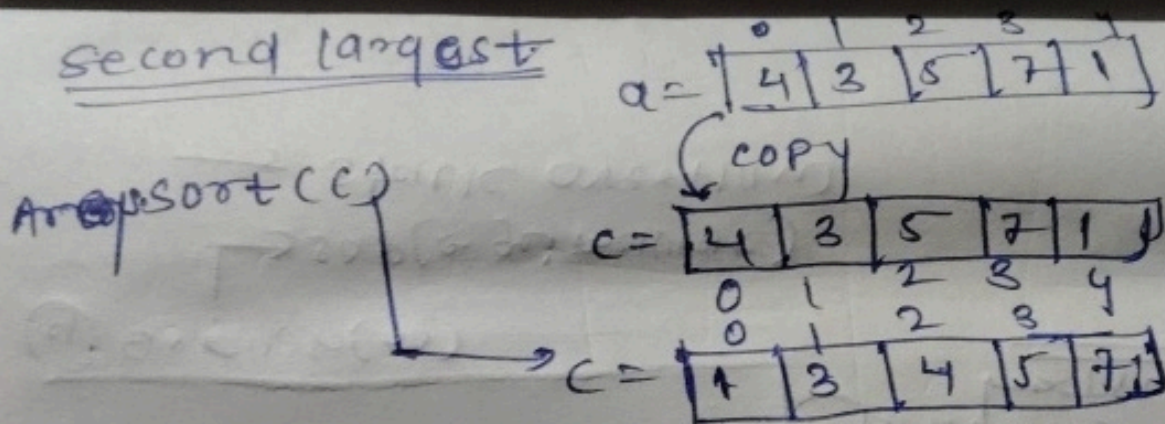
a[2] == 7

7 == 7 true

end P

return i ← found at 2

second largest



for (int i = c.length - 2; i >= 0; i--)

i = 5 - 2
i = 3

3 >= 0 i--

1) i = 3 3 >= 0

c[3] != c[5 - i]

c[3] = c[4]

5 != 7 - true

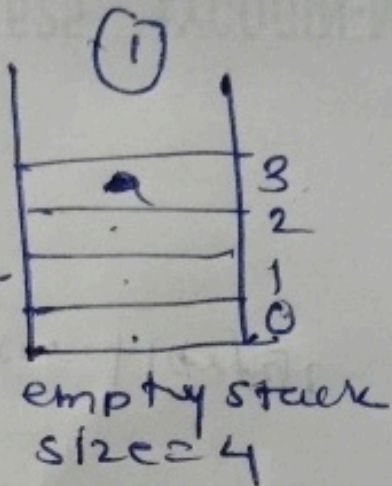
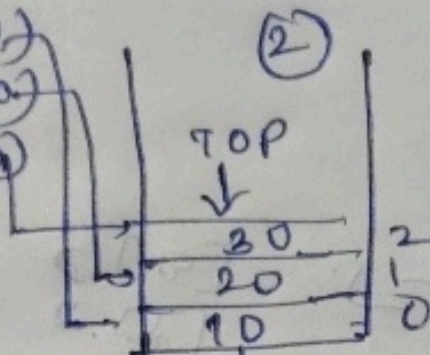
return c[i]

c[3] = 5 - second largest

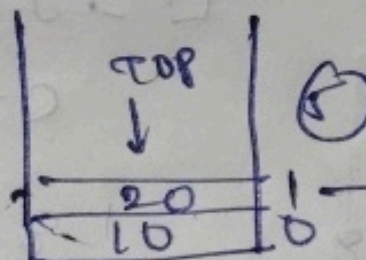
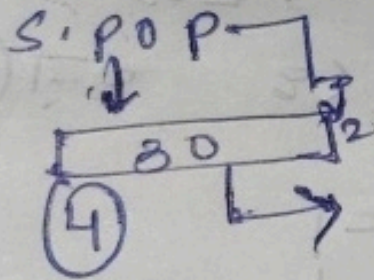
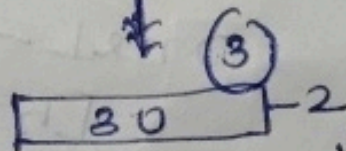
Arraystack

size = 4

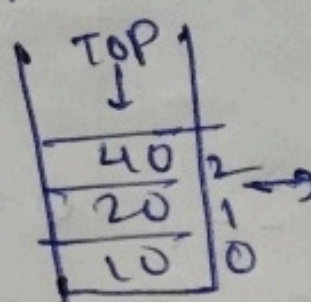
s.push(10)
s.push(20)
s.push(30)



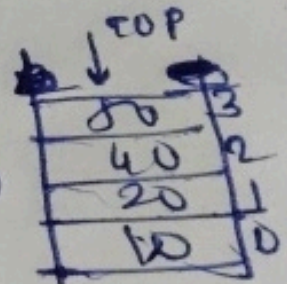
s.pop()



s.push(40)



s.push(50)



s.push(60)

return

s.push(60)

→ s.pop() → get message

→ stack overflow

Linear Queue

front = 0 rear = -1
capacity = 5

10	15	14	16	18
1	2	3	4	

Linear Queue

front = 0 rear = -1
capacity = 5

15	13	14	16	18
0	1	2	3	4
	↑ rear			rear

for empty

~~front~~

① front > rear

$0 > -1$ ✓

1) $x = 15$

2) $x = 13$

3) $x = 14$

4) $x = 16$

5) $x = 18$

② data[++rear] = x
data[0] = 15

③ data[front] = rear
 $0 > 0$

data[1] = 13

④ data[2] = 14

⑤ data[3] = 16

⑥ data[4] = 18

now rear = 4

is full() → rear == 5 - 1
5 - 1 = 4 true